# Soporte técnico de una red de oficinas

Sistema multiagente con reglas de seguridad en código · Alberto Olmos · Práctica de Laboratorio I, IA Agéntica · octubre 2026

## 1. Problema y alcance

El sistema hace de **soporte de primer nivel** para los empleados de cuatro oficinas (MAD-01, MAD-02, BCN-01, VAL-01). Entra el texto de la incidencia, la oficina y un ticket opcional; sale un **Resultado** con el estado final (*resuelto*, *escalado*, *rechazado* o *tope_alcanzado*), la respuesta al empleado con citas a la base de conocimiento y la traza completa. Lo que no se resuelve se escala a segundo nivel con un resumen de lo probado. Queda fuera lo que no es soporte de oficinas (se rechaza) y cualquier reparación física. Está escrito en Python 3.12 **sin frameworks de agentes**: el bucle y las reglas son código propio que puedo explicar y probar.

## 2. Arquitectura multiagente

| Agente | Decide | Salida (JSON validado) | Herramientas |
|---|---|---|---|
| clasificador | tipo y prioridad | tipo, prioridad, motivo | buscar_tickets |
| diagnóstico | causa probable | causa, servicio, evidencias | consultar_servicio, consultar_equipo, buscar_tickets |
| conocimiento | procedimiento | procedimiento, citas, accion | buscar_docs; *propone* reiniciar_servicio y cerrar_ticket |
| escalado | traspaso a nivel 2 | resumen, probado, prioridad | crear_escalado |

**Orquestación por traspaso**: clasificador → diagnóstico → conocimiento → resuelto o escalado. El soporte es secuencial (no se puede proponer una solución sin conocer la causa), así que un supervisor solo añadiría llamadas al modelo y puntos de fallo. Las excepciones son reglas fijas en código: ticket reabierto o avería en varias oficinas → escalado tras el diagnóstico; sin cita válida o con la acción rechazada → escalado.

**Estado y mensajes**: un `Estado` compartido guarda lo averiguado (tipo, prioridad, causa, servicio, citas, acciones, traza y pasos). Cada agente recibe un resumen JSON de ese estado, no la conversación entera: menos texto, menos créditos y menos confusión. En cada turno el modelo devuelve un único JSON (llamada a herramienta o salida final); el código valida el contrato y su prompt se genera desde el registro, con **solo** las herramientas permitidas a ese agente (mínimo privilegio).

## 3. Herramientas y conocimiento

Siete herramientas en un registro con **descripción** y marca de **reversible**; todas pasan por un único punto (`ejecutar`) que convierte cualquier fallo en `ErrorHerramienta`. Las dos irreversibles (reiniciar_servicio, cerrar_ticket) nunca se ejecutan dentro del bucle de un agente: conocimiento solo las propone. Los datos de la red se cargan en memoria, así que cada ejecución parte del mismo estado.

La recuperación es un **BM25 escrito a mano** sobre 15 documentos propios (11 artículos kb-* con Síntomas, Causa y Pasos, y 4 políticas: catálogo de servicios, escalado, reinicios y prioridades). Cada fragmento es una sección, así que las citas son precisas (`docs/kb-red-01.md#Pasos`). Lo elegí frente a embeddings porque no necesita red ni dependencias, es determinista y explicable. Limitación: no entiende sinónimos; se mitiga con frases de empleados en cada sección de síntomas.

## 4. Robustez y seguridad

| Regla (en código) | Qué garantiza |
|---|---|
| R1 Confirmación | Una acción irreversible solo se ejecuta si confirmar() devuelve verdadero. Si la persona dice que no, no se ejecuta y se escala. |
| R2 Guardarraíl | Un servicio crítico en horario laboral solo se reinicia si es el diagnosticado y está caído o degradado; nunca en otra oficina; máximo un reinicio por servicio. Si bloquea, ni siquiera se pregunta. |

| Regla (en código) | Qué garantiza |
|---|---|
| R3 Tope | 12 pasos (respuestas del modelo y herramientas): evita bucles y limita el gasto. Al superarlo, tope_alcanzado. |
| R4 Errores | Errores de herramienta, herramienta no permitida y JSON ilegible son observaciones; dos ilegibles seguidas → escalado. |
| R5 Citas | Una cita solo vale si ese fragmento lo devolvió buscar_docs en esta ejecución; las inventadas se borran. Sin cita válida no hay resuelto. |

Además: la prioridad la sube el código si los datos lo exigen (un crítico caído es P1 aunque el modelo diga P3) y nunca la baja; la respuesta al empleado la redacta una plantilla, así que el modelo no puede prometer plazos. En Claude Code, `settings.json` permite solo leer y probar, pide permiso para dependencias, ejecuciones que gastan créditos y commits, y prohíbe leer `.env` (también por terminal), borrar, publicar y sacar datos. El token vive solo en mi `.env`: no está en el código ni en la entrega.

## 5. Pruebas y evaluación

276 pruebas con **dobles del modelo** (ScriptedModel con respuestas fijadas y un cliente falso para Hugging Face): deterministas, sin red y sin token. Cada regla R1–R5 tiene pruebas que fallan si se rompe; se comprobó rompiendo cada regla a propósito (mutaciones).

La **evaluación** son 11 casos cuyo resultado esperado definí yo aplicando mis reglas, sin mirar el código (oráculo independiente). Mide el **resultado** (final, tipo, prioridad) y la **trayectoria** (C = clasificador, D = diagnóstico, K = conocimiento, E = escalado). Se ejecuta con guion (comprueba las reglas) y con el modelo simulado (lo que vería un usuario sin token).

| # | Caso | Esperado: final · tipo · prioridad · trayectoria |
|---|---|---|
| 1 | «no tengo internet», MAD-01, confirma (vpn caída) | resuelto · red · P1 · C-D-K |
| 2 | Igual, pero rechaza el reinicio (R1) | escalado · red · P1 · C-D-K-E |
| 3 | «no tengo internet», VAL-01 (tarjeta rota): mismo síntoma, otra causa | resuelto · red · P3 · C-D-K |
| 4 | Impresora, MAD-02, T-0007 reabierto | escalado · impresora · P4 · C-D-E |
| 5 | ERP caído en MAD-02 y BCN-01; el modelo dice P2 | escalado · acceso_aplicacion · P1 · C-D-E |
| 6 | Correo degradado, BCN-01; el modelo dice P4 | resuelto · correo · P2 · C-D-K |
| 7 | «¿me recomiendas un restaurante?» | rechazado · fuera_de_ambito · — · C |
| 8 | Oficina inexistente (SEV-01) | rechazado · — · — · sin agentes |
| 9 | Cita inventada por el modelo (R5) | escalado · red · P1 · C-D-K-E |
| 10 | Propone reiniciar el ERP con la vpn diagnosticada (R2) | escalado · red · P1 · C-D-K-E |
| 11 | «me va lento el ordenador» (caso propio) | resuelto · hardware · P4 · C-D-K |

**Primera pasada: 9/11 con guion y 6/9 con el simulado.** No cambié lo esperado; analicé cada fallo. Uno era del código (un rechazo conservaba la prioridad P4 del clasificador: lo que no se tramita no debe tener prioridad), otro de contenido (no había artículo de equipos lentos) y tres del simulado (no reconocía lo que está fuera de ámbito ni «ordenador», y empezaba siempre en P3). El caso 11 con el simulado **acertaba el final por casualidad**: «resuelto» citando el artículo de la tarjeta de red. Por eso se evalúa también la trayectoria y los datos. Tras los arreglos: **11/11 y 9/9**.

## 6. Dos trazas reales

**Traza 1.** «no tengo internet» en MAD-01 con el modelo real: pide confirmación del reinicio de la vpn.

> *Pendiente: se ejecutará con el modelo real (`--modelo hf --traza`) cuando la cuenta de Hugging Face tenga créditos.*

**Traza 2.** «no puedo entrar en el ERP» en MAD-02 con el modelo real: avería en varias oficinas, escalado P1.

> *Pendiente: se ejecutará con el modelo real (`--modelo hf --traza`) cuando la cuenta de Hugging Face tenga créditos.*

## 7. Forma de trabajo y limitaciones

Dirigí a Claude Code con un **CLAUDE.md** de menos de 200 líneas (contratos, reglas y valores cerrados), permisos de mínimo privilegio y **una orden por hito** con criterios comprobables (hitos 0 a 7b). Las decisiones son mías; Claude Code redactó a partir de ellas y generó el código, que revisé. El `worklog.md` recoge decisiones, problemas, comprobaciones y quién redactó cada cosa.

- El simulado funciona por palabras clave: una frase sin ninguna conocida puede clasificarse mal. Con el modelo real decide el clasificador.
- R5 comprueba que la cita se recuperó, no que sea relevante.
- BM25 no entiende sinónimos; con más documentos convendría una búsqueda híbrida.
- El tope de 12 pasos se ajustará tras medir el modelo real.

**Reproducir:** `uv sync` · `uv run pytest` · órdenes del README (modelo simulado por defecto, sin token).